TRACE-Eval: change-aware, cost-bounded regression test selection for LLM agent systems

Ciprian Păduraru
University of Bucharest
ciprian.paduraru@unibuc.ro

Abstract

An LLM agent system is assembled from skills, prompts, tools, harness code and configuration, and verified by a suite T of tests whose cost $c(t)$ is dominated by tokens. After a change Δ, a team either reruns all of T or guesses. We formulate selection, prioritization and stopping as one problem: choose tests from T, one at a time, under a budget B, so that the regressions Δ introduces are found with high probability at low cost. The change becomes a set of components $C(\Delta)$ (code lines and functions, skill procedures and prose, agent fields, configuration keys), each broken with a prior probability π_c. A test's sensitivity $s_{tc} = P(t \text{ fails} \mid c \text{ broken})$ is calibrated from the dependencies observed in execution traces and from the record of earlier changes, and a test that already fails on the unchanged system, with rate f_t, is discounted. The posterior over the $2^{|C(\Delta)|}$ subsets of broken components is kept exactly; the next test maximises the expected information gain per token; the run stops with REGRESSION at the first failure of a test that used to pass, with CLEAN when $P(\text{some component broken} \mid \text{outcomes}) < \varepsilon$, and not before every mandatory property Δ may affect has been tested. We evaluate on a testbed: a multi-agent content-operations application whose agents are played by an open model (Gemma 4 E4B), 176 tests, 43 controlled changes applied as file edits, and a ground truth measured per model by majority over three runs. Declared dependencies miss 39% of the regressions; trace-informed selection finds 86% within 20% of the full-suite tokens; two published methods reproduced on the testbed, CASPER's slices and a risk-aware bandit, find 50% and 59% because neither reads the change; the proposed method finds 79% for 3% of the tokens, 86% of the changes that break a safety property, and gives no false verdict on the 14 changes no test reveals.

1 What a team needs

1.1 The setting

A platform such as Adobe Experience Manager's agent layer ships dozens of agents. Each is a bundle of a system prompt, a list of tools, a list of skills (prompt files with a procedure the model follows), and settings, running on a shared harness that dispatches tool calls, retries, enforces approvals and delegates between agents. One prompt may cross several agents: an orchestrator writes a plan, runs each step with a specialist, passes the ids one step produced to the next, and asks a person before anything goes live. The platform is verified by scenario tests, which run a request end to end and check the final state and the trace, and by datasets of prompts with expected routings or tool calls. Every test is a run of a model. It costs tokens and seconds, and the cost varies by a factor of ten to a hundred between a one-shot routing item and a workflow across five agents.

After a change, the question is the one regression testing has always asked [33]: which tests to run (selection), in what order (prioritization), and when to stop. Three things make the classic answers [24, 25] hard to transfer. Much of the "code" is natural language, so coverage of a skill or a prompt is not a line of a control-flow graph. The outcome of a test is the behaviour of a model, so it is not a function of the code alone and can vary from run to run. And the cost of a

test is not one unit, so an order that finds a fault after three tests can still have spent a third of the budget.

1.2 The desired features

The AEM team, asked what they lack and want in a prototype, listed the following. Each is built in the testbed of this paper and shown where.

- **Selection, prioritization and stopping under a budget**, with a verdict a developer can act on (Section 4, the proposed method).
- **A catalog of properties**: the behavioural requirements the tests check, each with a category, how it is checked, a threshold on its pass rate and a flag that makes it a release blocker; and a way to draft the catalog from the requirements with a model, for people to edit (Section 2).
- **Coverage from several angles**: of the suite, per dimension (agents, features, skills, tools, tool error paths, configuration, properties, workflows, execution paths, risk), declared and observed; and of each change, the share of what it touches that the selected tests exercised, including whether every mandatory property it may affect got a test (Section 3).
- **Per-test metadata**: agents, skills, tools, configuration, workflow, risk, cost and history, declared by the author and observed in traces (Section 3).
- **Trace assertions**: final state, required and forbidden tools, attempts, audit entries, step limits, and the order of agents and of tools (Section 2).
- **Workflows** that run several agents from one prompt, with plans, hand-offs of ids between steps and human approvals, as a unit of testing and of coverage (Section 2).
- **Environment failures kept apart** from the agent's failures: a run the model server or the network broke never counts against the agent (Section 3).
- **OpenTelemetry traces**, with the plan, the routing decisions and the approvals as events [21] (Section 2).

1.3 What exists, and what is missing

Regression test selection and prioritization have a long literature [33]: safe selection on control-flow graphs [24], prioritization scored by APFD [7, 25] and its cost-cognizant form [6, 16], information retrieval over the diff [26], history and time windows in continuous integration [8, 17], learned selection [14, 28] and black-box diversity [22]. Ma et al. [13] name the obstacles for LLM applications: no crisp oracle, brittle prompts, nondeterminism. The literature on efficient evaluation of language models [10, 20, 23, 31] answers a different question: which subset of a benchmark reproduces a score. It selects once, for all models, and not after a change. GBV-PD [4] ranks tests from a parameter update, which needs gradients the models here do not expose. Fault injection for agents [27, 29] perturbs the runtime; the testbed here mutates the artefacts under change. The closest work, CASPER [19], is a regression method for LLM-based systems, but it is change-aware only through drift: it runs representatives of behavioural slices on the new version and ranks the slices by how much their outcomes moved. Risk-aware adaptive evaluation [15] allocates a budget of runs with a bandit, by impact and by the outcomes seen so far. Neither reads the diff. This paper's claim is that the diff, mapped to the tests through execution traces, is worth a lot, and that a sequential Bayesian treatment of the outcomes turns selection, prioritization and stopping into one decision.

The contributions are: a formulation of change-aware, cost-bounded testing of agent systems, with the metrics of regression testing adapted to token costs (Section 3); a method that extracts a change set at the granularity of code lines, skill procedures, agent fields and configuration keys, maps it to tests through dependencies observed in traces, ranks tests by expected information per token in a Bayesian model whose sensitivities are calibrated from history, and stops with a verdict (Section 4); a testbed with a multi-agent application, 176 tests, 43 controlled changes and a mutation-based ground truth derived per model, on which strategies are compared by offline replay (Section 2); and an evaluation against six baselines and two published methods reproduced on the testbed (Section 5).

2 How we built the testbed

This section is the informal account: what we built, how we wrote the tests and the properties, how the models run, and how we made the changes. The formal objects come in Section 3.

The application. We wanted an application that is small enough to read in an afternoon and shaped like the platforms the question comes from. ContentOps is a content-operations platform in the style of a web content manager: pages with versions, assets, publishing with reviews and schedules, link validation, SEO metadata, translation, governance with consent and approval tokens, rollback, brand rules, analytics and a site audit. Eleven agents run on a shared harness: an orchestrator and ten specialists, three of them (brand governance, data insights, site optimization) modelled on Adobe's. Each specialist follows a skill, a Markdown file with prose rules and a machine-readable procedure that the model reads as numbered steps, with the skills it includes written out and every setting shown with its value. Thirteen skills, 32 tools over deterministic in-memory services, and one configuration file make up the rest. The orchestrator delegates a request to one specialist or, when it needs several, writes a plan, hands each step its own instruction and the ids earlier steps reported, and asks a person before any step that publishes, schedules or takes pages down. The approver's answer is fixture data, so a run is repeatable.

The structure is deliberate. Some components are shared and have a wide blast radius: a governance skill that every writing specialist includes, a link-validation skill included by three, a permission check available to eight specialists, a retry helper around every tool call. Others are narrow: the approval gate matters to one test. Some rules live only in a skill (publish only with permission), others also in a tool or a service (approval tokens, roles). That is what makes selection hard, and what lets us discuss defence in depth: when a change removes a check from a skill, usually nothing below the skill catches the mistake.

The tests. We wrote the tests the way a team would write them for the application, before and independently of the changes. A scenario test gives an entry agent, a request in plain language ("Please publish the welcome page"), a context, a fixture and a fault profile (healthy, one transient failure of a tool, one lost acknowledgement, a tool permanently down). Its expectations are deterministic: the final state (the page is published, the audit log has one entry), the tools required and forbidden, attempt counts, facts the answer must report (a job id, which link is broken), a step limit, and, for the workflows, the order of agents and of tools. We check facts, not wording: a check that the answer contains "broken" would fail a model that says "invalid". Two cases with a rubric also have their answer judged by a model, only after the deterministic checks pass. Two datasets add items of the kind the AEM team runs nightly: a prompt with its expected delegation, and a prompt with its expected tool call. Fifteen tests run one of six multi-agent workflows end to end (a campaign launch across five specialists, a site optimization across four, a legal takedown, a localized price update, the rollback of a broken release, a privacy request), each with a variant where a step must stop the plan. Every test names its risk level,

the properties it checks, and the components its author declares it depends on. The declared block is incomplete on purpose, as it is in practice: nobody declares the shared skill reached through an include, the harness code or the configuration keys.

The properties. The AEM team wanted "prompt, properties and threshold", with categories. We wrote a catalog of fifteen properties in six categories (correctness, safety, resilience, orchestration, efficiency, quality), each with a description, how its tests check it (the final state, the trace, a judge, a dataset), a threshold on the pass rate of its tests, a flag that makes it a release blocker, and the components that state it. The six safety properties (authorization, guarded mutation, no broken publish, privacy, human approval, brand compliance) are mandatory with threshold one. Each test lists the properties it checks. We then had a model draft the same catalog from the requirements alone, the skills' rules, the agent definitions, the configuration guards and the specs, within fixed limits, and tag the tests; a person edits the draft and applies it. The model's catalog is good and traces every property to the component that states it; its tags of the tests need the person's review.

The environment. A real model plays the agents: Gemma 4 E4B, served with vLLM on a GPU server and decoding greedily, so that the same prompt gives the same answer up to the arithmetic of batching. Qwen3.8-27B and gpt-oss-120b serve as second and third models through an OpenAI-compatible endpoint, as they are served, without sampling. Every run is traced: the agents started, the skills loaded, the tools called with their status, the configuration keys read, and the code functions and lines executed, with a dynamic coverage context per test and repetition. A trace exports as OpenTelemetry spans, with the plan, the delegations and the approvals as span events. A run the environment broke, because the server or the network did not answer, is marked and left out, as if it had not been made. The whole suite runs K times on the current version and on every changed copy, and the strategies are compared afterwards by replay: a strategy that would have run a test gets the recorded outcome and cost of that test under that change, so thousands of comparisons cost nothing.

The changes. We wrote 43 controlled changes as file edits, in the spirit of mutation analysis [5, 9], each to resemble a regression a team would ship (Table 1): ten to tool adapters, services and the harness, three to the runtime loop, eleven to skills, two to agent definitions, seven to configuration, one combined change whose two halves are harmless alone, and nine benign changes meant to break nothing. Two rules shaped the set. A change has to touch an input the model acts on; a keyword weight or the order of an agent's intents is not one. And a removed step has to go from both the procedure and the prose of a skill, or the model does the step anyway. We left out what told nothing about selection: twins of other changes, crashes, and a change that broke most of the suite. Ten of the changes aim at the workflows. Which test reveals which change was measured afterwards, never designed, and no strategy is told.

3 Abstractions

Versions, tests, costs, budget. A system has a current version V_0 and a changed version $V_1 = V_0 + \Delta$. The suite T holds tests and eval items, each with a cost $c(t) > 0$ in tokens (prompt and completion tokens over every model call the test makes), a risk weight $\rho_t \in \{1, 2, 3, 4\}$, a set of properties $\mathrm{props}(t)$ from the catalog $\mathcal{P}$, and a declared dependency set D_t^{decl}. A budget B is a number of tokens, given as a fraction of the cost of the whole suite, $\sum_{t \in T} c(t)$.

Outcomes and the baseline. Running t on a version V gives an outcome $X_t(V) \in \{\mathrm{pass}, \mathrm{fail}\}$, which is random because a model produces it. A run the environment broke is not an outcome

Table 1: The 43 controlled changes.

category	count	examples
code	10	argument swap in a tool adapter; retries drop idempotency keys; approval gate bypassed; external links treated as reachable; a shared read reports the wrong review flag; ranking sorted the wrong way; case-sensitive search and brand check
runtime	3	the agent loop stops six steps early; a delegated answer is truncated; a delegation drops the ids of earlier steps
skill	11	publishing drops link validation or the permission check; authoring drops its metadata step; translation hard-codes a default; rollback restores the wrong version; governance drops the consent check; the routing skill stops asking for approval, or drops the brand gate
agent definition	2	an agent loses a tool
configuration	7	link validation disabled; review not required; a step budget of five; no retries; a title limit of twenty; another default agent; no approval needed to take a page down
combined	1	a keyword rule in code and a very low limit in configuration
benign	9	prose reflowed or reworded; a comment or docstring edited; local variables renamed; configuration comments reordered; an unused tool added; prompt wording; lists reordered

and is left out. K repetitions on V_0 give the baseline fail rate $f_t = \hat{P}(X_t(V_0) = \text{fail})$; a test with $f_t \geq \frac{1}{2}$ is one the model already fails, a mistake of the model rather than a fault of the system.

Components and the change set. A component is a part of the system that a change can touch and a run can exercise, named in one vocabulary: `code:<file>::<function>` for a function whose normalised syntax tree changed, `line:<file>:<n>` for a line of code, `tool:<name>` for a registered tool adapter, `skill:<name>#procedure`, `#prose` or `#meta` for the three parts of a skill file, `agent:<name>.<field>` for a field of an agent definition (a tool or skill list that only lost items names them), `config:<key>` for an effective setting, and `fixture:<name>`. Each component has a kind $k(c)$. The change set $C(\Delta)$ is the set of components the diff of V_0 and V_1 touches. Comments, docstrings, blank lines, formatting and the order of configuration mappings add nothing: no component, no file and no diff line, so no policy and no probability can depend on them. Skill files are prompts the model reads word for word, so all of their text counts.

Dependencies. The observed dependencies of a test come from its traced baseline runs. For a test t and a component d, $\phi_t(d) \in [0, 1]$ is the share of t's baseline runs that exercised d; $D_t^{\text{obs}} = \{d : \phi_t(d) > 0\}$. A component matches a dependency set when the set holds the component, its base (`skill:x` for `skill:x#procedure`), or a function of a module changed at module level. The required dependencies are the mandatory properties of $\mathcal{P}$. A test's behaviour signature is the tuple of agents, tools, fault profile and dataset of its baseline runs; tests with the same signature are near-duplicates.

Ground truth. A test t *reveals* Δ when its majority outcome flips: the pass rate on V_0 is at least $\frac{1}{2}$ and the pass rate on V_1 is below $\frac{1}{2}$, both over K repetitions. $\mathcal{R}(\Delta)$ is the set of revealing tests; Δ is a regression when $\mathcal{R}(\Delta) \neq \emptyset$. A test the model already fails cannot reveal anything. The ground truth therefore belongs to the model: a change the model cannot see, because it ignores the rule with and without the change, is revealed by no test under that model, and a different model may give a different $\mathcal{R}(\Delta)$. A property p is affected by Δ when $C(\Delta)$ touches one of its sources, or when a test that checks p exercised a changed component on V_0; $\mathcal{P}(\Delta)$ is the set of affected properties.

Policies. A policy sees $C(\Delta)$, the baseline facts about every test (D_t^{obs}, ϕ_t, $c(t)$, f_t, ρ_t, props(t), D_t^{decl} and the test's text), and the outcomes of earlier changes on which it may calibrate, but never $\mathcal{R}(\Delta)$. It chooses a test, observes its outcome and cost on V_1, and either chooses again or stops, within B. A non-adaptive policy is an order of T run greedily within the budget. An adaptive one may also return a verdict, CLEAN or REGRESSION, and a probability.

Metrics. For a regression, *detection* is whether a revealing test was run and failed, and the *cost to first detection* is the tokens spent up to that failure. *Tokens spent* is the cost of every test run, on regressions and on changes that break nothing. A *false alarm* is a failure of a test outside $\mathcal{R}(\Delta)$, which on a change that breaks nothing is a failure the change did not cause. A verdict is *correct* when it agrees with $\mathcal{R}(\Delta) \neq \emptyset$. *Change coverage*, which needs no ground truth, is the share of $C(\Delta)$ the executed tests exercised, the share of $\mathcal{P}(\Delta)$ they checked, whether every mandatory property in $\mathcal{P}(\Delta)$ got a test, and the shares of the affected workflows and features they reached. Prioritization alone, with every test run and no stopping, is scored by the average percentage of faults detected [25]. With $n = |T|$, $m = |\mathcal{R}(\Delta)|$ and TF_i the position in the order of the test that reveals fault i,

$$\text{APFD} = 1 - \frac{\sum_{i=1}^{m} TF_i}{n\,m} + \frac{1}{2n}, \tag{1}$$

which is near 1 when the first tests find every fault and near 0 when the last ones do; a random order scores about $\frac{1}{2}$. Because costs differ by an order of magnitude, we read its cost-cognizant form [6, 16]. With c_j the cost of the j-th test in the order, $C = \sum_j c_j$, and σ_i the severity of fault i (the risk weight of its revealing test),

$$\text{APFD}_c = \frac{\sum_{i=1}^{m} \sigma_i \left(C - \sum_{j<TF_i} c_j - \frac{1}{2} c_{TF_i}\right)}{C \sum_{i=1}^{m} \sigma_i}. \tag{2}$$

Finding a fault after a few cheap tests costs little and scores high; finding it after the long workflows scores low, however few tests came before. An order can score well on APFD and poorly on APFD_c by putting the expensive tests first.

4 The methods

Every method answers the same question, which tests to run on V_1 and in what order, from the inputs of a policy (Section 3). Figure 1 follows the proposed method from the diff to a verdict; the baselines use subsets of the same inputs. For each method we give its input, its output, its procedure and what it is good and bad at. Two reference orders need no procedure: `full` runs the suite in catalog order, `random` in a random permutation; both run greedily within the budget and never stop.

4.1 Selection on declared metadata: `declared`

Input: $C(\Delta)$; for every test, D_t^{decl}, ρ_t, $c(t)$. **Output:** an order of the selected tests. It is classic regression test selection on what the authors declared [24].

Algorithm 1 `declared`

1: $S \leftarrow \{t \in T : D_t^{\text{decl}}$ matches some $c \in C(\Delta)\}$
2: $a_t \leftarrow$ the number of components of $C(\Delta)$ that D_t^{decl} matches, for $t \in S$
3: **return** S sorted by ρ_t descending, then a_t descending, then $c(t)$ ascending

Good at: it is cheap, explainable, and safe when the metadata is complete. *Bad at:* it caps at what the authors declared; shared skills, harness code and configuration keys are reached without being declared, so it tops out whatever the budget.

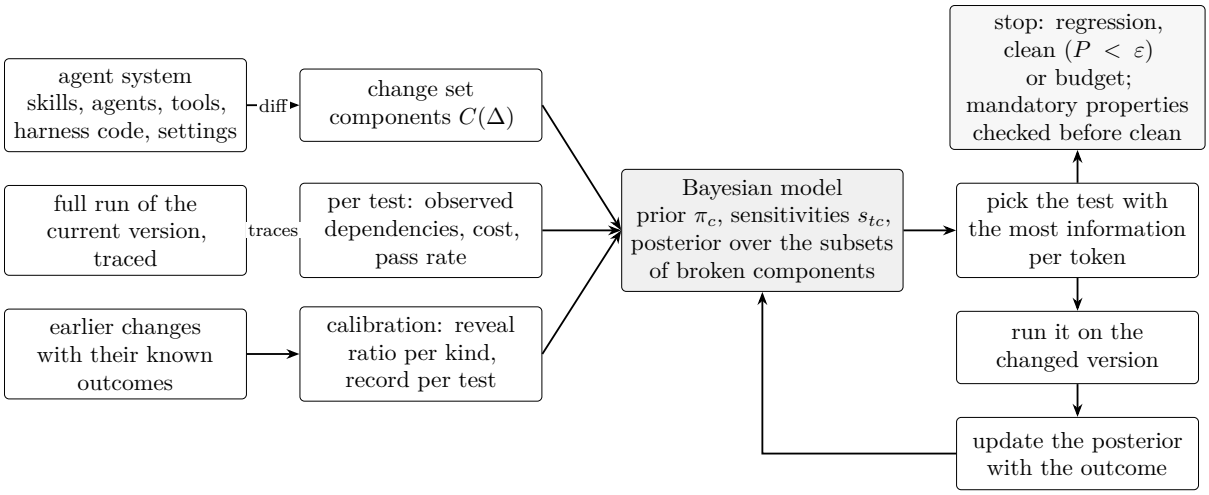


Figure 1: From a diff to a verdict. The three inputs on the left exist before the change is tested: the diff, a traced run of the current version, and earlier changes with their known outcomes. The loop on the right runs one test at a time on the changed version.

4.2 Text retrieval over the diff: `ir_tfidf`

Input: the text of the change (file names, component ids, the diff of the code) and the text of every test (name, request, expectations); $c(t)$. **Output:** an order of the whole suite. It is information-retrieval prioritization, as in REPiR [26], and uses no traces.

Algorithm 2 `ir_tfidf`

$q \leftarrow$ TF-IDF vector of the change text, over the vocabulary of the tests' texts
$v_t \leftarrow$ TF-IDF vector of the text of t, for every $t \in T$
return T sorted by $\cos(q, v_t)$ descending

Good at: it needs no instrumentation and is strong wherever tests, skills and settings share vocabulary, which is the common case in a textual system. *Bad at:* it never stops, so it spends the whole budget on every change, benign ones included, and it is blind to code reached without a shared word.

4.3 Selection on traces: `observed`

Input: $C(\Delta)$; for every test, D_t^{obs} with ϕ_t, ρ_t, $c(t)$. **Output:** an order of the selected tests. It is the trace-informed selection this paper argues for.

Algorithm 3 `observed`

$S \leftarrow \{t \in T : D_t^{\text{obs}}$ matches some $c \in C(\Delta)\}$
$a_t \leftarrow \sum_{c \in C(\Delta) \text{ matched}} \phi_t(c)$, with line hits counted once per file, for $t \in S$
return S sorted by $a_t\, \rho_t / c(t)$ descending

Good at: it selects everything the traces show the change can reach, so its recall of the revealing tests is complete at a fraction of the suite's cost, and it needs only one traced run per test. *Bad at:* it does not stop within its selection and treats a covered component as an affected one, so it spends on tests that execute the changed line without being able to notice the change.

4.4 Cost-aware ranking of the whole suite: `cost_aware`

Input: $C(\Delta)$; for every test, D_t^{obs} with ϕ_t, the baseline pass rate $p_t = 1 - f_t$, ρ_t, $c(t)$. **Output:** an order of the whole suite, in the spirit of cost-cognizant prioritization and of history-weighted continuous testing [6, 17].

Algorithm 4 `cost_aware`

1: $a_t \leftarrow$ affinity as in `observed` (zero for an unaffected test); $\lambda_t \leftarrow [t$ executed a changed line$]$
2: $\text{score}_t \leftarrow \rho_t \, (0.05 + a_t + \lambda_t) \, (2 - p_t)/c(t)$
3: **return** T sorted by score_t descending

Good at: affected tests first, cheap ones before expensive ones, and the rest after them so that the budget is always used. *Bad at:* it never stops, and its history factor $(2 - p_t)$ pushes forward the tests the model already fails, which it then reports as failures the change did not cause.

4.5 CASPER's slices, reproduced: `casper_slices`

Input: the baseline runs of every test (ten behavioural signals: turns, tool calls, distinct tools, the share of the most-called tool, the entropy of the calls, hand-offs, skills loaded, settings read, the size of the run and the share the model wrote) and the baseline outcome; $c(t)$; B. **Output:** a sequence of runs on V_1 (the representatives, then the slices in order). CASPER [19] mines signals from the logs of the current version, cuts the suite into slices that are coherent in signal space and consistent in outcome, runs a few representatives of every slice on the new version, and ranks the slices by how much their behaviour moved. Here the slices come from k-means [12] on the standardised signals with the baseline outcome as a heavier column, in place of the paper's evolutionary search, and, because the replay oracle returns an outcome and a cost rather than a trace, a slice is scored by the shift of its representatives' failure rate.

Algorithm 5 `casper_slices`

Input: baseline runs of every test; slice size n_s, minimum size $n_{\min}$, outcome weight w, representatives K_r; budget B
1: $Z \leftarrow$ the standardised behavioural signals of every test, with $w\cdot$ (standardised pass rate) as an extra column
2: slices $\leftarrow$ k-means$(Z, \lceil |T|/n_s \rceil)$; merge every slice smaller than $n_{\min}$ into the slice whose centre is nearest
3: **for all** slices j **do**
4: $\quad \text{reps}_j \leftarrow$ the member nearest the centre, then farthest-point sampling, K_r in all
5: **end for**
6: run the representatives of every slice on V_1, cheapest slices first, while the budget allows
7: $\text{shift}_j \leftarrow$ failure rate of reps_j on V_1 minus their baseline failure rate
8: order the slices: shift > 0 by size, then shift < 0, then unchanged
9: run the members of each slice in that order, nearest the centre first, until the budget is spent

Good at: it needs no diff, so it applies to a model upgrade, a provider's silent update or a prompt-level change in a system whose code the testing team does not own; its cost does not grow with the suite, since a fixed number of representatives per slice samples a large suite as cheaply as a small one; and the slices are a map of the suite's behaviours a team can read. *Bad at:* it is blind to an edit, so on a change with a diff it orders no better than random; it has no stopping rule; and a representative the model answers unstably drags its whole slice forward.

4.6 Risk-aware adaptive evaluation, reproduced: `risk_aware_ts`

Input: for every test a context vector x_t (features of its definition and of the shape of its baseline runs, standardised, with an intercept), an impact $\iota_t = \rho_t$, $c(t)$; B. **Output:** a sequence of runs on V_1, chosen one at a time. Risk-aware adaptive evaluation [15] is a contextual Thompson-sampling bandit [3, 30]: the reward of a trial is impact times failure; each round, a logistic regression on every outcome so far gives every scenario a failure probability, a Beta posterior per scenario mixes it with the scenario's own outcomes, and one draw per scenario, times the impact, picks the next. Here the scenarios are the tests, the budget is in tokens and each test runs once.

Algorithm 6 `risk_aware_ts`

Input: context x_t of every test, impact ι_t, costs $c(t)$; mixing weight λ, ridge penalty λ_2; budget B

$n_t \leftarrow 0$, $k_t \leftarrow 0$ for every t; $b \leftarrow B$
run a test chosen uniformly at random among those with $c(t) \le b$
while some unrun t has $c(t) \le b$ **do**
  $w \leftarrow \arg\min_w \sum_{t:\, n_t>0} \mathrm{logloss}(\sigma(w^\top x_t), k_t) + \frac{\lambda_2}{2}\|w\|^2$ ▷ Newton's method, warm start
  $p_t \leftarrow \sigma(w^\top x_t)$; $\theta_t \sim \mathrm{Beta}\big(1+\lambda p_t + k_t,\ 1+\lambda(1-p_t)+n_t-k_t\big)$ for every unrun t that fits
  run $t^\star \leftarrow \arg\max_t \iota_t\,\theta_t$ on V_1; $n_{t^\star} \leftarrow 1$; $k_{t^\star} \leftarrow$ [failed]; $b \leftarrow b - \kappa$
end while

Good at: it learns from the outcomes it sees and weighs them by impact, so it finds the high-risk failures first once a few have been seen. *Bad at:* any failure is a reward, so a test the model already fails counts as a find; it ignores the diff; with no outcome yet it has only the risk to go on; and it has no stopping rule and no verdict.

4.7 The proposed method: `adaptive_bayes`

Input: $C(\Delta)$ with the kind of each component; for every test D_t^{obs} with ϕ_t, D_t^{decl}, its text, f_t, ρ_t, $c(t)$, $\mathrm{props}(t)$ and its behaviour signature; the earlier changes $\mathcal{D}$ with their revealing sets; the catalog $\mathcal{P}$; B. **Output:** a sequence of runs on V_1, a verdict (REGRESSION, CLEAN or BUDGET) and the probability that some component is broken.

Hypotheses. Each changed component $c \in C(\Delta)$ is broken with a prior π_c that depends on its kind: about 0.45 to 0.5 for code, skill procedures, configuration and agent fields, 0.12 to 0.15 for prompt wording and skill prose. Tool and line components are views of a code change, not separate causes, and are folded into it. The posterior is kept exactly over all 2^k subsets $h \in \{0,1\}^{C(\Delta)}$, with $k \le 8$ (the likeliest components are kept when there are more).

Likelihood. A test passes only if it would pass on the current version and none of the broken components makes it fail:

$$q_t(h) = P(t \text{ fails} \mid h) = 1 - (1 - f_t) \prod_{c:\, h_c=1} (1 - s_{tc}). \tag{3}$$

A test the model already fails (f_t near 1) fails under every hypothesis, so its outcome carries no information, which is what keeps the method from mistaking the model's mistakes for regressions.

Sensitivity. Covering a component is weak evidence by itself: most tests that execute a changed function do not fail when it breaks. The sensitivity is therefore calibrated (Algorithm 8) from the earlier changes whose outcomes are known, never from the change under test:

$$s_{tc} = \mathrm{clip}\big(r_{k(c)} \cdot \ell_{t,k(c)} \cdot \phi_t(c) \cdot \tau_t,\ 0.02,\ 0.9\big), \tag{4}$$

where r_k is the reveal ratio of the component's kind k (among the tests that statically looked impacted by earlier changes of that kind, the share that revealed them, with the kinds separating a test that executed a changed line from one that only called the changed function, and agent fields calibrated per field); $\ell_{tk} = (O_{tk} + m)/(E_{tk} + m)$ is the test's own lift on that kind, with O_{tk} the earlier changes of kind k it revealed, E_{tk} the number an average impacted test would have revealed (the sum of those changes' reveal rates), and $m = 2$ a padding that keeps a short record from moving the lift far; $\phi_t(c)$ is how often the baseline runs exercised the component, or, for an agent that lost tools or skills, how often t used one of the lost items; and $\tau_t \in [0.5, 2]$ tilts the sensitivity up for a test whose text shares vocabulary with the diff and for one whose author declared the dependency. The lift matters because a change that breaks most of the suite says little about each test it breaks, while a change that one test alone caught says a lot about that test. A test with no link to the component gets a small constant $s_{\text{ind}} = 0.02$; one declared but never observed gets half the kind's ratio.

Policy, stopping and the mandatory rule. Algorithm 7 runs the loop. The next test maximises the expected reduction of the posterior's entropy [11], tilted by risk, per token. After a pass, tests with the same behaviour signature as the one that passed have their sensitivities discounted by $\delta = 0.35$, because near-duplicates give correlated evidence rather than independent draws. Each test runs at most once: the models decode greedily, so a rerun would almost always repeat the answer. The run stops with REGRESSION at the first failure of a test that passed on the current version, because under greedy decoding that failure is a changed behaviour, which is what the ground truth calls revealing. It stops with CLEAN when $1 - P(h = \mathbf{0}) < \varepsilon$, with $\varepsilon = 0.05$, and otherwise when the budget is spent or no test can still move the posterior. Before a run ends without a regression, every mandatory property in $\mathcal{P}(\Delta)$ gets a test if it has none yet: the cheapest test of that property that passes on the current version, while the budget allows. A failure there is a regression like any other.

Algorithm 7 `adaptive_bayes`: adaptive Bayesian selection with stopping

Input: components $C(\Delta)$ with priors π_c; tests T with costs $c(t)$, fail rates f_t, risk tilts ρ_t and sensitivities s_{tc} (Algorithm 8); near-duplicate groups; budget B; ε, δ
Output: the tests run, a verdict, and $1 - P(\mathbf{0})$

$P(h) \propto \prod_c \pi_c^{h_c}(1-\pi_c)^{1-h_c}$ for $h \in \{0,1\}^{C(\Delta)}$ ▷ $h = \mathbf{0}$: nothing is broken
$q_t(h) \leftarrow 1 - (1-f_t)\prod_{c:\, h_c=1}(1-s_{tc})$ for every t and h ▷ Equation 3
$R \leftarrow \emptyset$; $b \leftarrow B$
loop
  if $1 - P(\mathbf{0}) < \varepsilon$ **then**
    return FINISH(CLEAN)
  end if
  $p_t \leftarrow \sum_h P(h)\, q_t(h)$ ▷ $P(t$ fails$)$ under the posterior
  $g_t \leftarrow H(P) - p_t\, H(P \mid t \text{ fails}) - (1-p_t)\, H(P \mid t \text{ passes})$ ▷ expected information gain
  $t^\star \leftarrow \arg\max\{\, g_t\, \rho_t / c(t) : t \notin R,\ c(t) \le b,\ g_t > 0 \,\}$
  if there is no such $t^\star$ **then**
    return FINISH(BUDGET)
  end if
  run $t^\star$ on V_1: outcome x, cost κ; $R \leftarrow R \cup \{t^\star\}$; $b \leftarrow b - \kappa$
  $P(h) \leftarrow P(h)\, q_{t^\star}(h)$ if $x =$ fail, else $P(h)\,(1 - q_{t^\star}(h))$; normalise
  if $x =$ fail and $f_{t^\star} < \frac{1}{2}$ **then**
    return REGRESSION ▷ a test that used to pass has failed
  end if
  if $x =$ pass **then**
    $s_{uc} \leftarrow \delta\, s_{uc}$ and recompute q_u for every unrun u in the group of $t^\star$
  end if
end loop

procedure FINISH(verdict)
  for all mandatory properties $p \in \mathcal{P}(\Delta)$, in id order **do**
    if no test in R checks p **then**
      $u \leftarrow$ the cheapest $u \notin R$ with $p \in \text{props}(u)$, $f_u < \frac{1}{2}$, $c(u) \le b$; **continue** if none
      run u; $R \leftarrow R \cup \{u\}$; $b \leftarrow b - \kappa$; update P as above
      if u failed **then**
        return REGRESSION
      end if
    end if
  end for
  return verdict, with $1 - P(\mathbf{0})$
end procedure

Algorithm 8 Calibration of the sensitivities from the earlier changes

Input: earlier changes $\mathcal{D}$ with their revealing sets $\mathcal{R}(\Delta')$; defaults r_k^0 per kind; padding m; the change under test $\Delta \notin \mathcal{D}$

Output: s_{tc} for every test t and component $c \in C(\Delta)$

for all $\Delta' \in \mathcal{D}$ and components $c' \in C(\Delta')$ of kind k **do**
  $I \leftarrow$ the tests that look impacted by c': executed a changed line, called the changed function, or match the component
  rate $\leftarrow |\mathcal{R}(\Delta') \cap I|/|I|$; append rate to A_k
  O_{tk} += $[t \in \mathcal{R}(\Delta')]$ and E_{tk} += rate for $t \in I$ ▷ once per change and kind
end for
$r_k \leftarrow \mathrm{clip}\big((\sum A_k + m\, r_k^0)/(|A_k| + m),\ 0.01,\ 0.9\big)$ for every kind k
for all tests t and components $c \in C(\Delta)$ of kind k **do**
  $\ell_{tk} \leftarrow (O_{tk} + m)/(E_{tk} + m)$; $\tau_t \leftarrow 0.5 + \mathrm{text}(t, \Delta) + 0.5\,[\text{declared}]$
  $s_{tc} \leftarrow \mathrm{clip}(r_k\, \ell_{tk}\, \phi_t(c)\, \tau_t,\ 0.02,\ 0.9)$ if $\phi_t(c) > 0$; $\frac{1}{2} r_k$ if declared only; s_{ind} otherwise
end for

Good at: selection, prioritization and stopping come from one model, so the method returns the budget it does not need, gives a verdict with a probability, discounts the tests the model already fails, and tests every mandatory property the change may affect before it calls the change clean. *Bad at:* a regression that only one to five tests reveal, where it can become confident that nothing broke before reaching them; it needs a traced baseline and a record of earlier changes to calibrate on; and its stopping rule assumes greedy decoding. For the prioritization metrics, which score a complete order, the method's order is the greedy sequence it would pick if every test passed, with no stopping, followed by the untouched tests in cost-aware order.

5 Evaluation

Questions. RQ1: can the tests relevant to a change be identified, and what do declared and observed dependencies each give? RQ2: can the tests be ordered so that regressions are found earlier? RQ3: how much budget can be returned while detection and coverage hold, and does the stopping rule give the right verdict? RQ4: what do the two published methods, which do not read the change, find on the same testbed?

Protocol. The full suite runs $K = 3$ times on the current version with coverage contexts, which gives the observed dependencies, costs and pass rates, and $K = 3$ times on each of the 43 changed copies, which gives the ground truth. Every strategy is then replayed offline: for every change, budget in $\{2, 5, 10, 20, 30, 50, 100\}\%$ of the full-suite tokens and five seeds, the strategy sees the baseline facts and the change set, and each test it runs is answered from a recorded run of that changed copy. The method calibrates on the other 42 changes, never on the one under test.

Ground truth. On the current version 170 of 176 tests pass. The six failures are the model's own mistakes, which clearer wording did not fix: it schedules a page without an approved review, requests a review nobody asked for, applies an over-long title, goes on to publishing after a failed link check, and stops a whole plan after a refused step the other steps do not depend on. Of the 34 regressions, 29 are revealed by at least one test. The model does not act on the five remaining guards (a review flag, a length limit, a non-blocking asset reference) with or without the change, so it behaves the same, wrongly, on both versions. No benign change fails a test. Across the 43 changes the three runs disagreed on five (change, test) pairs, which the majority vote absorbs. The 29 revealed regressions and the 14 changes no test reveals are the population below.

Table 2: Detection of the 29 revealed regressions by nominal budget (share of the full-suite tokens), and the tokens actually spent: on regressions at a 20% and a 100% budget, and on the benign changes at 100%. Five seeds; the last three columns are shares of the full-suite tokens.

strategy	2%	5%	10%	20%	50%	100%	spent@20	spent@100	benign@100
`full`	0.00	0.14	0.21	0.34	0.90	0.99	0.20	0.98	1.00
`random`	0.17	0.21	0.33	0.52	0.84	0.99	0.20	0.98	1.00
`declared`	0.45	0.55	0.55	0.61	0.61	0.61	0.07	0.10	0.13
`ir_tfidf`	0.59	0.70	0.71	0.92	0.99	0.99	0.20	0.98	0.99
`observed`	0.48	0.62	0.69	0.86	0.86	0.99	0.15	0.49	0.32
`cost_aware`	0.38	0.59	0.62	0.83	0.86	0.99	0.20	0.98	0.99
`casper_slices`	0.17	0.28	0.43	0.50	0.88	0.99	0.20	0.98	1.00
`risk_aware_ts`	0.12	0.27	0.43	0.59	0.81	0.99	0.20	0.98	1.00
`adaptive_bayes`	0.59	0.76	0.79	0.79	0.83	0.83	0.03	0.03	0.20

Table 3: Order quality and verdicts. APFD and APFD_c of the complete order; cost to the first true detection as a share of the full suite (detected cases); detection of the 21 regressions that break a safety property at a 10% and a 20% budget; share of replays, over all budgets, in which the strategy saw a failure on one of the 14 changes no test reveals.

strategy	APFD	APFD_c	first detection	safety@10	safety@20	false alarm
`full`	0.63	0.57	0.19	0.10	0.24	0.71
`random`	0.56	0.53	0.14	0.36	0.57	0.63
`declared`	0.81	0.76	0.025	0.52	0.55	0.51
`ir_tfidf`	0.78	0.77	0.040	0.79	0.98	0.70
`observed`	0.75	0.79	0.048	0.57	0.81	0.57
`cost_aware`	0.75	0.78	0.056	0.57	0.76	0.91
`casper_slices`	0.59	0.54	0.12	0.50	0.60	0.57
`risk_aware_ts`	0.60	0.61	0.13	0.55	0.73	0.71
`adaptive_bayes`	0.83	0.81	0.017	0.86	0.86	0.00

Selection (RQ1). Declared metadata caps detection at 61% however large the budget (Table 2), because shared skills reached through an include, harness code and most configuration keys are not declared. The observed dependencies recover full selection recall at half the suite's cost and reach 86% within a 20% budget. The argument for trace-informed selection holds.

Prioritization (RQ2). The informed strategies raise APFD_c from 0.53 (random) to 0.76–0.81 (Table 3). The adaptive method's order is the best and finds the first regression soonest, after 1.7% of the suite's tokens. The two reproduced methods order no better than random (0.54 and 0.61) and find their first regression after 12–13% of the suite, where the change-aware strategies need 2–6%.

Budget and verdicts (RQ3). The adaptive method is the only strategy that returns budget. It finds 76% of the regressions at a 5% budget and 79% at 20%, for under 3% of the tokens, and spends a tenth of the suite on a benign change at a 20% budget (Figure 2). TF-IDF over the diff is the strongest selector at 20% (92%) and the strongest on the safety-breaking changes (98%), and it spends the whole budget every time, on benign changes too. The adaptive method's ceiling is 83%: on a regression that only one to five tests reveal it can become confident that nothing broke before reaching them (145 of 1,015 replays of regressions ended CLEAN). The changes it misses at 20% are the delegation that drops the ids of earlier steps, the loop that stops six steps early, retries without idempotency keys, the routing skill without its brand gate, the authoring skill without its metadata step, and the combined change, which runs out of budget.

The six tests that fail on the current version fail on every changed copy too. A strategy that

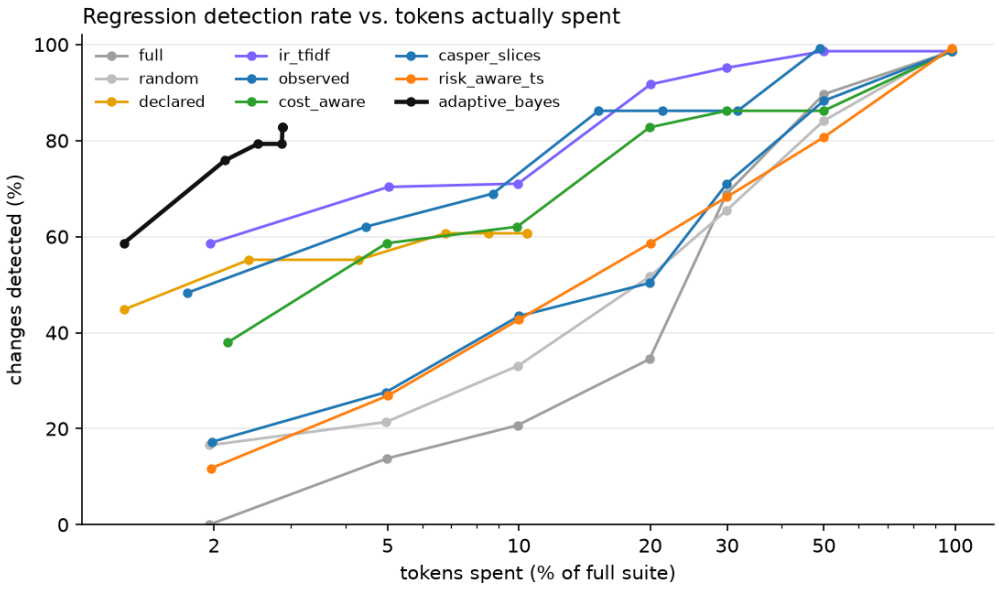


Figure 2: Detection of the 29 revealed regressions against the tokens actually spent, every strategy and budget. The adaptive method sits at the left: it stops.

Table 4: Detection by the method with three text measures, with the tokens spent in parentheses, against `ir_tfidf` alone.

budget	word overlap	TF-IDF	no text	`ir_tfidf` alone
5%	0.76 (2.1%)	0.79 (2.0%)	0.62 (2.4%)	0.70 (5%)
20%	0.79 (2.9%)	0.83 (2.4%)	0.69 (3.5%)	0.92 (20%)
50%	0.83 (2.9%)	0.83 (2.4%)	0.69 (4.5%)	0.99 (50%)

runs one sees a failure the change did not cause: on the 14 changes no test reveals, this happens to `full` in 71% of its replays, to `cost_aware` in 91%, to the bandit in 71% and to CASPER's slices in 57% (Table 3). The adaptive method discounts a test by its failures on the current version (Equation 3) and stops only on a test that used to pass, so it gave no REGRESSION verdict on any of these changes in 490 replays.

The text factor. The tilt τ_t can use word overlap (the default), the TF-IDF similarity of `ir_tfidf`, or no text at all (Table 4). TF-IDF brings forward the one test that reveals a narrow change and gains up to four points while spending less; without any text the method loses ten points. Text matters, and coverage and history carry most of the result. The testbed is textual, since tests, skills and configuration keys share vocabulary, which favours any text measure.

The mandatory rule. Without the rule, a run that found no regression had tested every mandatory property the change may affect in 33% of the replays at a 20% budget; with it, in 90%, for 0.8 extra tests per change on average, and detection rose from 76% to 79%. The rule is also a detector: it catches the publishing skill without its link validation, which the model alone calls clean, because the method must test the mandatory property "no broken publish" before ending, and that test fails. The rule never caused a false verdict. Below a 10% budget it often cannot afford the extra test.

The reproduced methods (RQ4), and where CASPER fits. CASPER's slices and the risk-aware bandit find 50% and 59% of the regressions at a 20% budget, between random (52%)

and the change-aware strategies (83–92%). The bandit's impact is the test's risk, so it does better on the changes that break a safety property (73% at 20%, 94% at 50%), and it rewards any failure, so it runs the model's red tests early. Neither stops. These numbers measure what the diff is worth, not CASPER in its own setting. CASPER was built for changes that have no diff a test can be mapped to: a model upgrade, a provider's silent update, a prompt-level change whose effect spreads through the model's behaviour. A platform sees both kinds of change. The change-aware strategies cover the edits to skills, code and settings; CASPER's representatives are the natural first pass after a model update, when every test is affected and the question is which behaviours drifted. The two could be combined: the slices can serve as the near-duplicate groups of the adaptive method, and a slice whose representatives drifted can enter the model as a changed component. On this testbed every change is an edit with a diff and the suite is small (176 tests, twelve slices), which is the case slicing has least to offer.

Other models. The same application was recorded with Qwen3.8-27B and with gpt-oss-120b, served as they are, with greedy decoding. Neither recording is a clean ground truth. Qwen passes 171 of the 176 tests on the current version and reveals every regression, but it also "reveals" every benign change, by two to five tests, including a change whose change set is empty: six tests answer differently when they run 64 at a time, as the changed copies do, than one at a time, as the baseline does, and its three runs disagreed on 260 (change, test) pairs against Gemma's five. vLLM's batch-invariant mode, which would make greedy decoding exactly repeatable, does not support the model's attention layers, and recording the baseline the way the changes are recorded leaves seven of the nine benign changes still flipping tests. gpt-oss-120b, recorded with five repetitions, varies more: the benign changes flip two to five tests under the majority over five runs and one to four under the first three, so more repetitions do not settle the vote; thirteen tests carry the variation, mostly long workflows that pass one at a time on the current version and fail in a batch on nearly every changed copy, whichever the change. A larger model is not automatically a cleaner ground truth, and a pipeline should measure how much its model's answers vary under the serving setup the selection will run on before trusting a ground truth or a stopping rule. The empty change measures exactly that.

Threats to validity. The regressions are mutants, not real changes; they were written to resemble regressions a team would ship, and the testbed does not yet pair them with a version history. Which test reveals which change was measured, not designed, and no strategy is told, but the tests and the changes were written by the same people. One primary application, built for the purpose, and one model with a stable ground truth: the second and third models show that the ground truth depends on the model and on how it is served. The testbed is textual, which favours the text factor and `ir_tfidf`; a second application with differently worded tests and changes (tau-bench [1, 32]) is planned. The calibration uses the other recorded changes, so leakage is controlled by leaving the change under test out, but the 42 training changes are few, which is why every learned ratio is padded toward a default. The stopping rule assumes greedy decoding; with sampling, a single failure would need repetitions [2, 18].

6 Take-aways

What the experiments say, in the order a team would act on them. The numbers are Gemma's, at a budget of 20% of the suite's tokens unless said otherwise.

1. **Record traces; do not maintain a map.** Hand-written dependency metadata finds 61% of the regressions at any budget, because shared skills, harness code and settings are reached without being declared. Dependencies read from traces find 86%, and everything at 100%. One instrumentation of the agent runtime replaces a test-to-component map that nobody keeps current.

2. **Read the diff.** The strategies that use the change find 79% to 92% of the regressions; the two that do not, CASPER's slices and the risk-aware bandit, find 50% and 59%, near a random order's 52%, and their first revealing test comes after 12% to 13% of the suite's tokens instead of 2% to 6%.

3. **Stop when confident, with a safety net.** The adaptive method finds 79% for 3% of the tokens and returns the rest. Its ceiling is 83%: on a regression that one to five tests reveal it can stop too early. The mandatory rule covers the safety properties for under one extra test per change, and on this application catches the publishing skill without its link check, which the stopping rule alone misses. The catalog of properties is where that policy lives: whoever sets the mandatory flag decides what the pipeline never skips.

4. **Discount the tests the model already fails.** Six tests fail on the current version, and on every changed copy. On the 14 changes no test reveals, every other strategy reports a failure the change did not cause in 51% to 91% of its replays; the adaptive method in none, because it keeps each test's pass rate on the current version and acts only on a flip.

5. **Measure the model's noise before trusting a verdict.** Gemma's three runs agree on every test of the current version and disagree on five (change, test) pairs over the 43 changes; Qwen's disagree on 260 and gpt-oss's on 577, both "reveal" every benign change, and five repetitions do not settle gpt-oss's vote. The baseline runs one test at a time and the changed copies in batches, and that alone flips some tests. An empty change, recorded like the others, measures the noise floor; record it per model and per serving setup.

6. **CASPER and the bandit have their place.** CASPER needs no diff: a model or provider update, or a prompt-level change in a system the team does not own. Its representatives are the natural first pass after a model update, and its slices could serve as the adaptive method's groups of near-duplicate tests. The bandit learns from the outcomes of earlier changes and from the tests it has run; with no history it has only the tests' risk to go on. The change-aware strategies cover edits to skills, code and settings; the two kinds combine.

7. **Text is a strong, cheap baseline.** TF-IDF similarity between the diff and the tests' text has the best detection when the whole budget is spent (92%, and 98% on the changes that break a safety property). It never stops, so it spends everything on benign changes too; it is the fallback order to keep in any pipeline.

8. **Not yet covered.** Real version history instead of mutants; a second system under test; suites large enough for slicing to pay; re-planning after a failed step; tuning the stopping threshold, the priors and the discount; time rather than tokens as the budget.

7 Conclusion and future work

After a change to an agent system, the diff is worth reading. Mapped to tests through the dependencies observed in traces, it finds 86% of the regressions within a fifth of the suite's tokens where hand-written metadata finds 61%, and the two closest published methods, which do not read it, find 50% and 59%. A Bayesian model over the changed components, with sensitivities calibrated from the record of earlier changes, turns selection, prioritization and stopping into one decision: it finds 79% of the regressions for 3% of the tokens, orders the suite best, gives no false verdict on the changes no test reveals, and tests every mandatory property a change may affect before it calls the change clean. The price is a ceiling on narrow regressions, which the mandatory rule lowers for the properties that matter most.

Three things come next. A second system under test, tau-bench's airline domain [1, 32], with the local model playing both the agent and the user, to see whether the text factor and the

calibration hold where tests and changes are worded differently. Real version histories beside the mutants, with the changes a team shipped and the tests that caught them, so that the reveal ratios are learned from practice. And the repetitions that a model with varying answers needs, with the empty change as the instrument: how many runs, under which serving setup, before a majority vote is a ground truth and a single failure is a verdict.

References

[1] Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. τ^2-bench: evaluating conversational agents in a dual-control environment, 2025. arXiv:2506.07982.

[2] Bjarni Haukur Bjarnason, André Silva, and Martin Monperrus. On randomness in agentic evals, 2026. arXiv:2602.07150.

[3] Olivier Chapelle and Lihong Li. An empirical evaluation of Thompson sampling. In *Advances in Neural Information Processing Systems 24 (NeurIPS)*, 2011.

[4] Eunho Cho, Donghwan Shin, and In-Young Ko. Evolution-aware regression test prioritization of ML-enabled systems using gradient-based behavior vectors, 2026. arXiv:2606.28037.

[5] Richard A. DeMillo, Richard J. Lipton, and Frederick G. Sayward. Hints on test data selection: help for the practicing programmer. *IEEE Computer*, 11(4):34–41, 1978.

[6] Sebastian Elbaum, Alexey G. Malishevsky, and Gregg Rothermel. Incorporating varying test costs and fault severities into test case prioritization. In *Proceedings of the 23rd International Conference on Software Engineering (ICSE)*, pages 329–338, 2001. doi: 10.1109/ICSE.2001.919106.

[7] Sebastian Elbaum, Alexey G. Malishevsky, and Gregg Rothermel. Test case prioritization: a family of empirical studies. *IEEE Transactions on Software Engineering*, 28(2):159–182, 2002. doi: 10.1109/32.988497.

[8] Sebastian Elbaum, Gregg Rothermel, and John Penix. Techniques for improving regression testing in continuous integration development environments. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, pages 235–245, 2014. doi: 10.1145/2635868.2635910.

[9] Yue Jia and Mark Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011.

[10] Peiyu Li, Xiuxiu Tang, Si Chen, Ying Cheng, Ronald Metoyer, Ting Hua, and Nitesh V. Chawla. Adaptive testing for LLM evaluation: a psychometric alternative to static benchmarks, 2025. arXiv:2511.04689.

[11] Dennis V. Lindley. On a measure of the information provided by an experiment. *The Annals of Mathematical Statistics*, 27(4):986–1005, 1956.

[12] Stuart P. Lloyd. Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982.

[13] Wanqin Ma, Chenyang Yang, and Christian Kästner. (Why) is my prompt getting worse? Rethinking regression testing for evolving LLM APIs. In *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering: Software Engineering for AI (CAIN)*, 2024. arXiv:2311.11123.

[14] Mateusz Machalica, Alex Samylkin, Meredith Porth, and Satish Chandra. Predictive test selection. In *Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 91–100, 2019. doi: 10.1109/ICSE-SEIP.2019.00018.

[15] Priyanath Maji and Spandan Ghose Chowdhury. Risk-aware adaptive evaluation: finding high-impact failures under limited budgets, 2026. URL `https://arxiv.org/abs/2609.38914`. arXiv:2609.38914; NeurIPS 2026 workshop on the evaluation of interactive agents.

[16] Alexey G. Malishevsky, Joseph R. Ruthruff, Gregg Rothermel, and Sebastian Elbaum. Cost-cognizant test case prioritization. Technical Report TR-UNL-CSE-2006-0004, University of Nebraska–Lincoln, 2006.

[17] Atif Memon, Zebao Gao, Bao Nguyen, Sanjeev Dhanda, Eric Nickell, Rob Siemborski, and John Micco. Taming Google-scale continuous testing. In *Proceedings of the 39th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 233–242, 2017. doi: 10.1109/ICSE-SEIP.2017.16.

[18] Evan Miller. Adding error bars to evals: a statistical approach to language model evaluations, 2024. arXiv:2411.00640.

[19] Biruk Asmare Muse, Lionel Briand, Yiwei Lu, and Keheliya Gallaba. CASPER: change-aware slice prioritization for efficient regression testing of LLM-based systems, 2026. URL `https://arxiv.org/abs/2608.00378`. arXiv:2608.00378.

[20] Franck Ndzomga. Efficient benchmarking of AI agents, 2026. arXiv:2603.23749.

[21] OpenTelemetry. Semantic conventions for generative AI: agent spans. `https://github.com/open-telemetry/semantic-conventions-genai/blob/main/docs/gen-ai/gen-ai-agent-spans.md`, 2026.

[22] Rongqi Pan, Taher A. Ghaleb, and Lionel C. Briand. LTM: scalable and black-box similarity-based test suite minimization based on language models. *IEEE Transactions on Software Engineering*, 50(11), 2024. arXiv:2304.01397.

[23] Felipe Maia Polo, Lucas Weber, Leshem Choshen, Yuekai Sun, Gongjun Xu, and Mikhail Yurochkin. tinyBenchmarks: evaluating LLMs with fewer examples. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. arXiv:2402.14992.

[24] Gregg Rothermel and Mary Jean Harrold. A safe, efficient regression test selection technique. *ACM Transactions on Software Engineering and Methodology*, 6(2):173–210, 1997. doi: 10.1145/248233.248262.

[25] Gregg Rothermel, Roland H. Untch, Chengyun Chu, and Mary Jean Harrold. Prioritizing test cases for regression testing. *IEEE Transactions on Software Engineering*, 27(10):929–948, 2001. doi: 10.1109/32.962562.

[26] Ripon K. Saha, Lingming Zhang, Sarfraz Khurshid, and Dewayne E. Perry. An information retrieval approach for regression test prioritization based on program changes. In *Proceedings of the 37th International Conference on Software Engineering (ICSE)*, pages 268–279, 2015. doi: 10.1109/ICSE.2015.47.

[27] Zahra Seyedghorban, Egor Klimov, Arie van Deursen, Annibale Panichella, and Burcu Kulahcioglu Ozkan. Observability and fault injection for LLM-based multi-agent systems in software engineering, 2026. arXiv:2608.24271.

[28] Helge Spieker, Arnaud Gotlieb, Dusica Marijan, and Morten Mossige. Reinforcement learning for automatic test case prioritization and selection in continuous integration. In *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 12–22, 2017. doi: 10.1145/3092703.3092709.

[29] Gou Tan, Zhensu Sun, Jieke Shi, Ting Zhang, Zilong He, Qingfu Wu, Shuai Liang, Weifeng Sun, Junda He, Pengfei Chen, Chuanfu Zhang, Lwin Khin Shar, and David Lo. AgentChaos: chaos engineering for agent systems via programmatic fault injection. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2026. arXiv:2608.06790.

[30] William R. Thompson. On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. *Biometrika*, 25(3–4):285–294, 1933.

[31] Skyler Wu, Yash Nair, and Emmanuel J. Candès. Efficient evaluation of LLM performance with statistical guarantees, 2026. arXiv:2601.20251.

[32] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: a benchmark for tool-agent-user interaction in real-world domains, 2024. arXiv:2406.12045.

[33] Shin Yoo and Mark Harman. Regression testing minimization, selection and prioritization: a survey. *Software Testing, Verification and Reliability*, 22(2):67–120, 2012. doi: 10.1002/stvr.430.